

POSTECH
TECH CHALLENGE — FASE 3

ToggleMaster

Infraestrutura como Código, DevSecOps e GitOps na AWS

Relatório de Entrega

PARTICIPANTES

Cristiano Alves de Oliveira Schaarschmidt

cristiano.schaarschmidt@gmail.com

Expedito Ferreira da Silva Filho

expedito.ads@gmail.com

Emily Marcolino

emily.marcolino@gmail.com

Setembro de 2026

Sumário

Sumário	2
1. Introdução	3
2. Infraestrutura como Código (Terraform)	4
3. Pipeline de CI/CD e DevSecOps	5
4. Entrega Contínua e GitOps	6
5. Decisões Técnicas	7
6. Desafios Encontrados	8
7. Estimativa de Custos da AWS	10
8. Conclusão	11
9. Links de Entrega	12

1. Introdução

A DevOps Solutions Inc. aprovou a arquitetura de microsserviços do ToggleMaster desenvolvida na Fase 2, mas a operação manual se tornou insustentável: desenvolvedores rodavam `kubectl apply` a partir de máquinas locais, gerando conflitos de versão; credenciais de banco de dados eram distribuídas em arquivos de texto sem segurança; uma vulnerabilidade em uma biblioteca Go passou despercebida até produção; e recriar o ambiente de homologação levava dias por ter sido montado manualmente no console da AWS.

A partir da premissa "se não está no código, não existe", esta fase substitui integralmente o provisionamento manual por Infraestrutura como Código (Terraform), adota um pipeline de integração contínua com práticas de DevSecOps — build, lint, varredura de dependências (SCA), varredura de código-fonte (SAST) e varredura de imagem de contêiner, com bloqueio automático em caso de vulnerabilidade crítica — e passa a realizar a entrega contínua via GitOps, com o ArgoCD sincronizando automaticamente o cluster EKS a partir de um repositório de manifestos Kubernetes.

O ambiente utilizado é o AWS Academy (Opção A), o que implica uma restrição relevante: o Terraform não pode criar Roles ou Policies de IAM, devendo referenciar a LabRole já existente na conta para o cluster EKS e os Node Groups. Essa condição, detalhada na seção 6, orientou diversas decisões de arquitetura ao longo do projeto.

A Figura 1 resume a arquitetura de ponta a ponta implementada: os módulos Terraform à esquerda provisionam a infraestrutura; o pipeline de CI/CD do GitHub Actions ao centro aplica as práticas de DevSecOps e publica as imagens; e o ambiente AWS à direita hospeda o cluster EKS, onde o ArgoCD mantém os cinco microsserviços (auth, flag, targeting, evaluation, analytics) sincronizados com o repositório GitOps.



Figura 1 — Arquitetura de ponta a ponta: IaC (Terraform), CI/CD (GitHub Actions) e AWS Cloud (EKS + ArgoCD).

2. Infraestrutura como Código (Terraform)

Toda a infraestrutura é declarada em três módulos Terraform — networking, eks e data-stores — com o estado remoto armazenado em um bucket S3 versionado, protegido por lockfile para evitar apply concorrente. Como o ambiente é o AWS Academy, o cluster EKS e os Node Groups não recebem uma role criada pelo Terraform: um data source lê a LabRole já existente na conta e associa seu ARN a ambos os recursos, garantindo zero recursos de IAM criados pelo projeto.

Ao todo, mais de vinte e cinco recursos são provisionados por código: a VPC com subnets públicas e privadas em duas zonas de disponibilidade, o Internet Gateway, o NAT Gateway e as route tables; o cluster EKS e seu node group; três instâncias RDS PostgreSQL (auth, flag, targeting); um cluster ElastiCache Redis com criptografia em trânsito; uma tabela DynamoDB (ToggleMasterAnalytics); uma fila SQS; e cinco repositórios ECR, um por microsserviço.

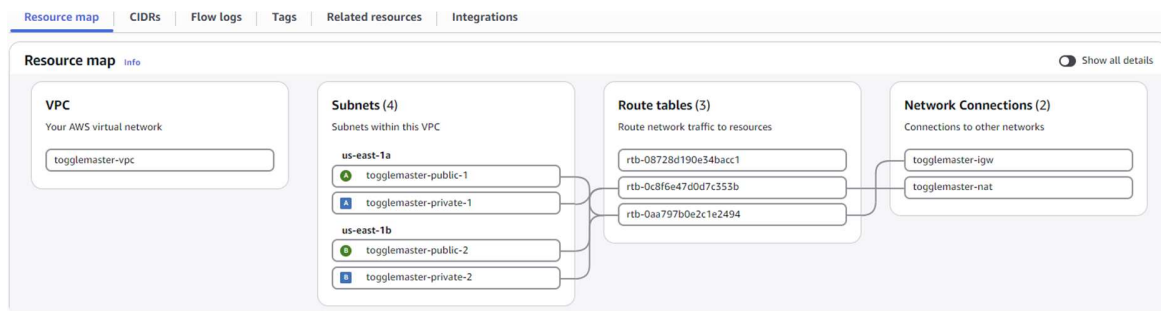


Figura 2 — Mapa de recursos de rede (VPC, subnets, route tables e conexões) provisionado via Terraform, visualizado no console da AWS.

```
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

cluster_endpoint = "https://B6CB8C520371DD8D79CC573D2FD9FF80.gr7.us-east-1.eks.amazonaws.com"
cluster_name = "togglemaster-cluster"
database_urls = <sensitive>
dynamodb_table = "ToggleMasterAnalytics"
ecr_repository_urls = {
  "analytics" = "982599766065.dkr.ecr.us-east-1.amazonaws.com/togglemaster-analytics"
  "auth" = "982599766065.dkr.ecr.us-east-1.amazonaws.com/togglemaster-auth"
  "evaluation" = "982599766065.dkr.ecr.us-east-1.amazonaws.com/togglemaster-evaluation"
  "flag" = "982599766065.dkr.ecr.us-east-1.amazonaws.com/togglemaster-flag"
  "targeting" = "982599766065.dkr.ecr.us-east-1.amazonaws.com/togglemaster-targeting"
}
rds_endpoints = {
  "auth" = "togglemaster-auth-db.c56xqyce9iqw.us-east-1.rds.amazonaws.com:5432"
  "flag" = "togglemaster-flag-db.c56xqyce9iqw.us-east-1.rds.amazonaws.com:5432"
  "targeting" = "togglemaster-targeting-db.c56xqyce9iqw.us-east-1.rds.amazonaws.com:5432"
}
redis_url = "rediss://master.togglemaster-redis.vofmox.use1.cache.amazonaws.com:6379"
sqs_queue_url = "https://sqs.us-east-1.amazonaws.com/982599766065/togglemaster-events"
```

Figura 3 — Saída do terraform apply, sem alterações pendentes, com os outputs dos recursos provisionados (cluster, ECR, RDS, Redis e SQS).

3. Pipeline de CI/CD e DevSecOps

Cada um dos cinco microsserviços possui um workflow de GitHub Actions, disparado em Pull Requests e em pushes para a main, reutilizando dois workflows genéricos (go-ci.yml e python-ci.yml) via workflow_call. O pipeline executa, em sequência, build e testes unitários, lint (golangci-lint ou flake8), varredura de dependências com Trivy (SCA), varredura de código-fonte com gosec ou bandit (SAST), e por fim o build e push da imagem Docker para o ECR, com a tag do commit hash.

A regra de bloqueio de segurança é o núcleo do DevSecOps implementado: caso o Trivy ou o SAST encontrem uma vulnerabilidade classificada como CRÍTICA, o estágio falha com exit code 1 e o pipeline é interrompido antes de a imagem ser construída ou publicada. A Figura 5 mostra exatamente esse comportamento em ação, com o job security-scan bloqueando o pipeline ao detectar três vulnerabilidades CRÍTICAS na dependência pyyaml.

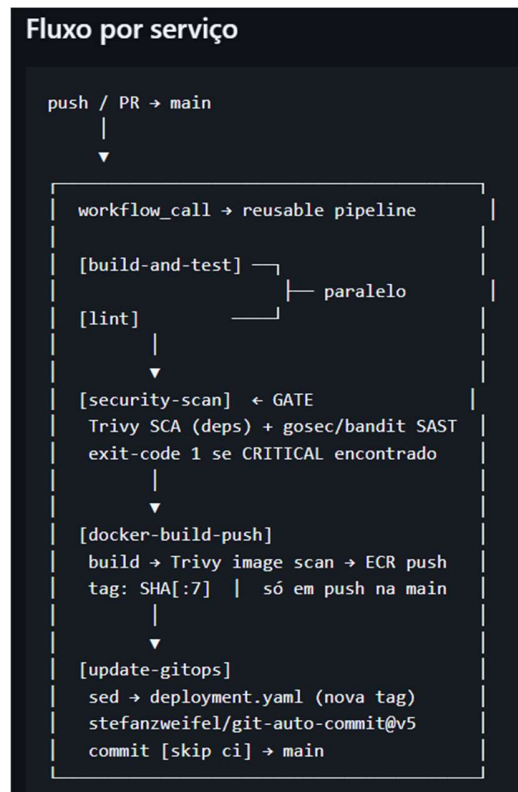


Figura 4 — Pipeline CI/CD (GitHub Actions) - Fluxo por microsserviço

The screenshot shows the GitHub Actions interface for the repository 'emilymarc / togglemaster-fase3'. The left sidebar contains navigation links: Code, Issues, Pull requests, Actions (selected), Projects, Security and quality, Insights, and Settings. Under the 'Actions' tab, there's a 'New workflow' button and a list of workflows: analytics-service, auth-service, evaluation-service, flag-service, reusable-go-ci, reusable-python-ci, and targeting-service. The main area is titled 'All workflows' and shows 'Showing runs from all workflows'. A 'Filter workflow runs' search bar is present. Below this, there's a 'Help us improve GitHub Actions' section. The '88 workflow runs' section displays a list of runs with columns: Workflow, Event, Status, Branch, and Actor. The runs include:

- Security scan fix (flag-service #19: Pull request #7 opened by emilymarc) - security-scan-fix - 9 minutes ago - 1m 34s
- Update requirements.txt security scan (flag-service #18: Pull request #6 opened by emilymarc) - security-scan - 15 minutes ago - 26s
- Merge branch 'main' of https://github.com/emilymarc/togglemaster-fase3 (targeting-service #17: Commit 6ed602 pushed by emilymarc) - main - 45 minutes ago - 1m 18s
- Merge branch 'main' of https://github.com/emilymarc/togglemaster-fase3 (auth-service #18: Commit c4580f pushed by emilymarc) - main - 47 minutes ago - 2m 16s
- Merge branch 'main' of https://github.com/emilymarc/togglemaster-fase3 (analytics-service #18: Commit 4136709 pushed by emilymarc) - main - 50 minutes ago - 1m 43s
- pipeline: update workflow (evaluation-service #18: Commit 1a1837c pushed by emilymarc) - main - 54 minutes ago - 1m 59s
- pipeline: update workflow (flag-service #17: Commit 8cae9c4 pushed by emilymarc) - main - Today at 05:02 - 1m 23s

Figura 5 — Execuções dos workflows de CI/CD no GitHub Actions para os cinco microsserviços do ToggleMaster e teste de security scan (Trivy).

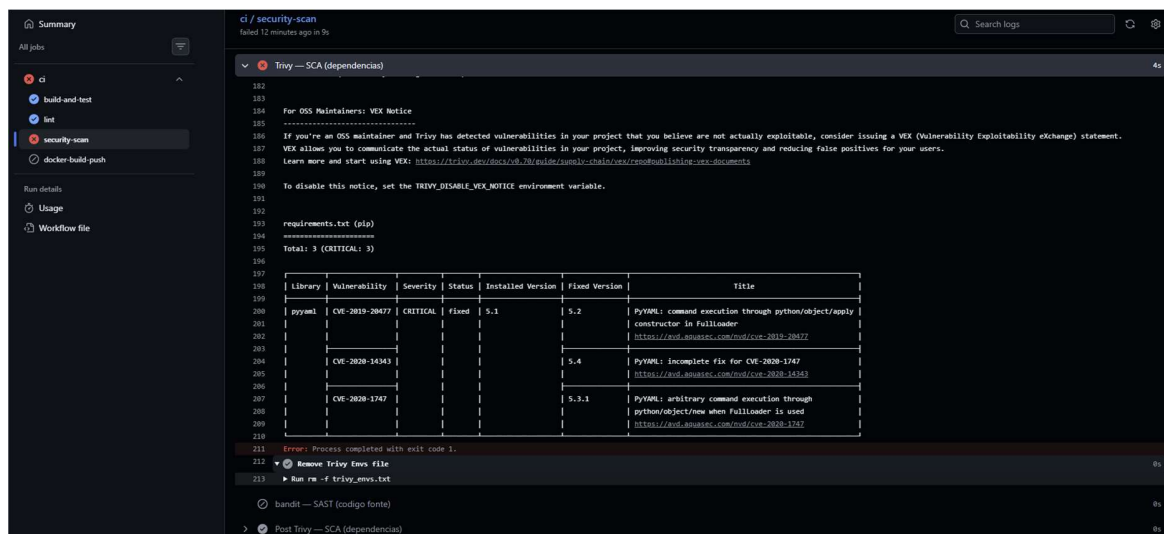


Figura 6 — Pipeline bloqueado pelo estágio security-scan (Trivy) ao detectar vulnerabilidades CRÍTICAS na dependência pyyaml, impedindo o avanço para o build da imagem.

4. Entrega Contínua e GitOps

O deploy abandona o push direto a partir do CI e adota GitOps: os manifestos Kubernetes de cada microsserviço vivem em uma pasta dedicada (gitops/services/<serviço>), e o ArgoCD, instalado dentro do próprio cluster EKS, monitora esse repositório e sincroniza automaticamente qualquer mudança. O pipeline de CI não possui nenhuma credencial do cluster — ao final do job docker-build-push, um passo atualiza a tag da imagem no deployment.yaml correspondente e comita a mudança de volta no repositório com a action stefanzweifel/git-auto-commit-action, deixando o ArgoCD responsável por aplicar o rollout.

A Figura 6 mostra as cinco Applications do ArgoCD — auth, flag, targeting, evaluation e analytics — todas com status Healthy e Synced, configuradas com auto-sync, self-heal e prune, refletindo automaticamente qualquer commit no repositório de GitOps sem intervenção manual.

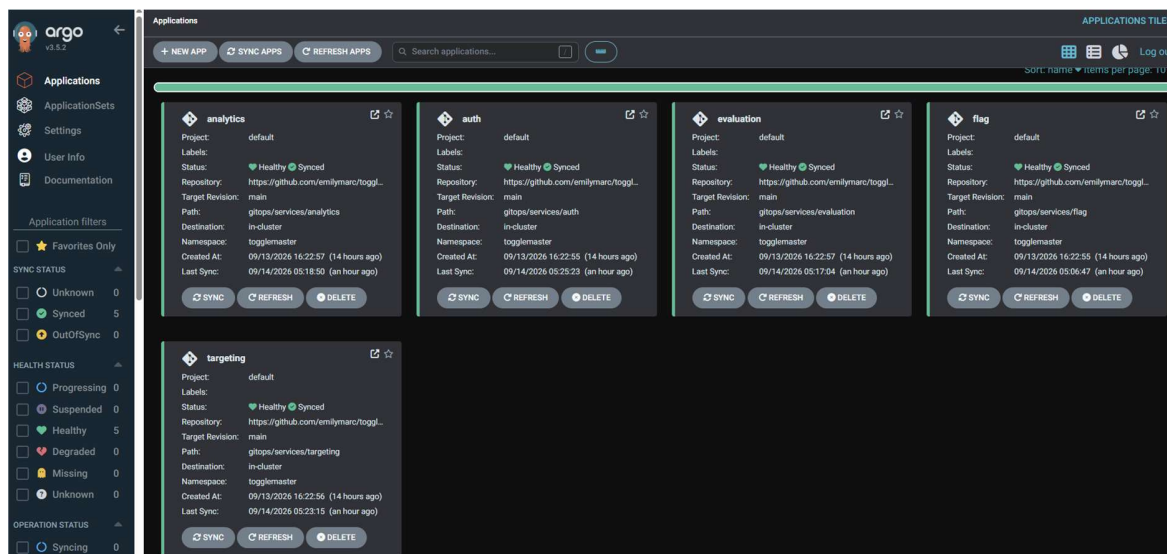


Figura 7 — Interface do ArgoCD com os cinco microserviços sincronizados automaticamente a partir do repositório GitOps.

5. Decisões Técnicas

As decisões abaixo documentam as escolhas de arquitetura mais relevantes tomadas ao longo do desenvolvimento, com a motivação técnica por trás de cada uma.

5.1. LabRole via data source (não criada pelo Terraform)

AWS Academy proíbe criar IAM Roles/Polices pelo Terraform. Solução: data "aws_iam_role" "lab_role" lê a LabRole já existente e a referencia diretamente no cluster EKS e nos Node Groups. Nenhuma permissão nova é criada.

5.2. Backend Terraform em S3 com lockfile no DynamoDB

Nenhum terraform.tfstate local commitado. O lockfile no DynamoDB evita apply simultâneo. Bucket S3 criado manualmente antes do primeiro terraform init (bootstrap obrigatório).

5.3. errcheck desabilitado via .golangci.yml por serviço

golangci-lint v1.64+ habilita errcheck por padrão. Os handlers HTTP da Fase 2 usam json.Encoder.Encode() sem checar o retorno — erros pós-WriteHeader() são irreversíveis. Decisão: criar .golangci.yml por serviço desabilitando apenas errcheck, sem tocar no código da Fase 2.

5.4. Reusable Workflows (workflow_call)

Um único go-ci.yml e um único python-ci.yml são reutilizados pelos 5 microserviços via workflow_call. secrets: inherit repassa os secrets do repositório. Isso evita duplicação de centenas de linhas de YAML.

5.5. stefanzweifel/git-auto-commit-action@v5 em vez de git manual

O job update-gitops precisa fazer git config, add, commit, pull --rebase e push de forma segura dentro do GitHub Actions. Em vez de escrever esses cinco comandos shell com tratamento de erro, usou-se a action stefanzweifel/git-auto-commit-action@v5, que encapsula todo o boilerplate e só cria o commit se o arquivo de fato mudou.

5.6. permissions: contents: write no job docker-build-push

Por padrão, o GitHub Actions concede ao GITHUB_TOKEN permissão de leitura nos workflows que usam workflow_call. Para que o bot consiga fazer push do commit de atualização de tag no repositório GitOps, é necessário declarar permissions: contents: write explicitamente no job. Sem isso, qualquer operação de escrita no Git resulta em erro HTTP 403.

6. Desafios Encontrados

O maior condicionante do projeto foi o ambiente de treinamento AWS Academy (Learner Lab), que opera com privilégios reduzidos e não permite a criação de recursos de IAM via Terraform — restrição institucional, não um erro de configuração, que moldou boa parte das decisões da seção anterior. Os demais desafios surgiram durante o desenvolvimento e a integração dos módulos de infraestrutura, dos pipelines de CI/CD e do fluxo de GitOps, e estão registrados abaixo.

6.1. Restrição de IAM do AWS Academy

O AWS Academy bloqueia a criação de IAM Roles/Políticas via Terraform, com erro de permissão. Resolvido usando um data source para a LabRole existente na conta. Exige conhecer o nome exato da role disponível no ambiente.

6.2. Versões fixas nos módulos Terraform

Módulos da comunidade sem version pin causam falhas silenciosas quando a versão mais recente quebra a API.

Solução: fixar versões explícitas no bloco required_providers e nos módulos.

6.3. depends_on explícito entre módulos

O Terraform não infere automaticamente dependências entre módulos que não compartilham variáveis. O cluster EKS precisa existir antes do node group; o RDS precisa do security group da VPC. Foi necessário adicionar depends_on explícito.

6.4. CVEs CRITICAL no scan de imagem Docker

O Trivy encontrou CVEs CRITICAL no pacote perl-base da imagem alpine. A flag ignore-unfixed: true não resolvia o problema porque os fixes já estavam disponíveis. Solução: migrar para distroless/static-debian12, que não contém nenhum pacote de SO.

6.5. golangci-lint habilitando errcheck por padrão na v1.64

A versão 1.64 do golangci-lint passou a habilitar errcheck por padrão sem arquivo de configuração, quebrando o CI com código legítimo da Fase 2. Resolvido criando .golangci.yml por serviço, sem alterar o código original

6.6. Variáveis de módulo vs. variáveis de ambiente no workflow

Segredos da AWS (Access Key, Secret Key, Region) precisavam ser injetados como env no GitHub Actions. O script set-github-secrets.sh automatiza esta tarefa e lê os valores do aws configure e os sobe para o repositório via gh secret set. As credenciais rotacionam a cada sessão do Academy.

6.7. io/ioutil depreciado desde Go 1.19

O services/evaluation/evaluator.go usava ioutil.ReadAll() para ler o body de respostas HTTP — package depreciado desde Go 1.16 e removido do lint em versões mais recentes do golangci-lint. Solução: substituir o import "io/ioutil" por "io" e as chamadas ioutil.ReadAll por io.ReadAll.

6.8. CVE-2025-68121 no stdlib Go — crypto/tls

O Trivy detectou CVE CRITICAL no stdlib do Go embutido no binário compilado — mesmo usando imagem runtime distroless (sem pacotes OS), o scanner inspeciona a versão do stdlib dentro do executável. O builder usava golang:1.21-alpine (v1.21.13), afetado por falha de validação de certificado TLS durante session resumption. Fix disponível a partir de Go 1.24.13. Solução: atualizar o builder para golang:1.24-alpine. Go é retrocompatível — código escrito para 1.21 compila sem alterações com 1.24.

6.9. CVEs fixáveis em python:3.11-slim

Os três serviços Python (flag, targeting, analytics) usavam python:3.11-slim com pacotes Debian desatualizados contendo CVEs com fix disponível. O ignore-unfixed: true não resolvia. Solução: adicionar apt-get upgrade -y no Dockerfile logo após apt-get update, atualizando todos os pacotes do SO para as versões com patch antes de copiar a aplicação.

6.10. Terraform state lock preso + bucket S3 do backend destruído

Após um terraform destroy mal-sucedido, o bucket S3 que armazena o estado remoto foi removido junto com a infraestrutura. O terraform apply passou a falhar com no such host ao tentar acessar o backend, e a entrada de lock no DynamoDB permaneceu travada com um UUID órfão. Solução: recriar o bucket e a tabela DynamoDB manualmente (aws s3 mb + aws dynamodb create-table), restaurar o estado salvo localmente e para liberar o lock antes de repetir o apply.

6.11. ImagePullBackOff — imagens tagueadas com SHA, deployment usa :latest

O CI fazia push das imagens ECR com a tag do commit curto (ex.: ad2fc75), mas os deployment.yaml referenciavam :latest, que não existia no ECR. O Kubernetes tentava puxar a tag :latest, causando ImagePullBackOff em todos os pods. Solução imediata: aws ecr put-image para copiar o manifest existente de cada imagem adicionando a tag :latest, seguido de kubectl rollout restart deployment -n togglemaster. Solução definitiva: adicionar o job update-gitops no CI para atualizar a tag no deployment.yaml a cada push.

6.12. Branch protection (GH013) bloqueando push do bot de CI

O job update-gitops falhou com GH013: Changes must be made through a pull request porque a branch main tinha uma regra de proteção exigindo PR para qualquer push direto — inclusive o push do bot do GitHub Actions. Solução: Settings → Rules → editar a regra → Bypass list → adicionar o ator GitHub Actions. Alternativa mais simples: Settings → Actions → General → Workflow permissions → marcar Read and write permissions, que concede write ao GITHUB_TOKEN de forma global.

6.13. Imutabilidade de tag ECR ao reexecutar a pipeline

Os repositórios ECR foram criados com image_tag_mutability = "IMMUTABLE". Ao fazer re-run de um job com o mesmo commit SHA, o push da imagem falha com "tag invalid: The image tag 'abc1234' already exists". Não é um bug — é a proteção de imutabilidade funcionando. Solução para re-run: criar um novo commit (git commit --allow-empty -m "ci: force new pipeline run") para gerar um SHA diferente. Em produção isso nunca ocorre, pois cada push de código produz um SHA único.

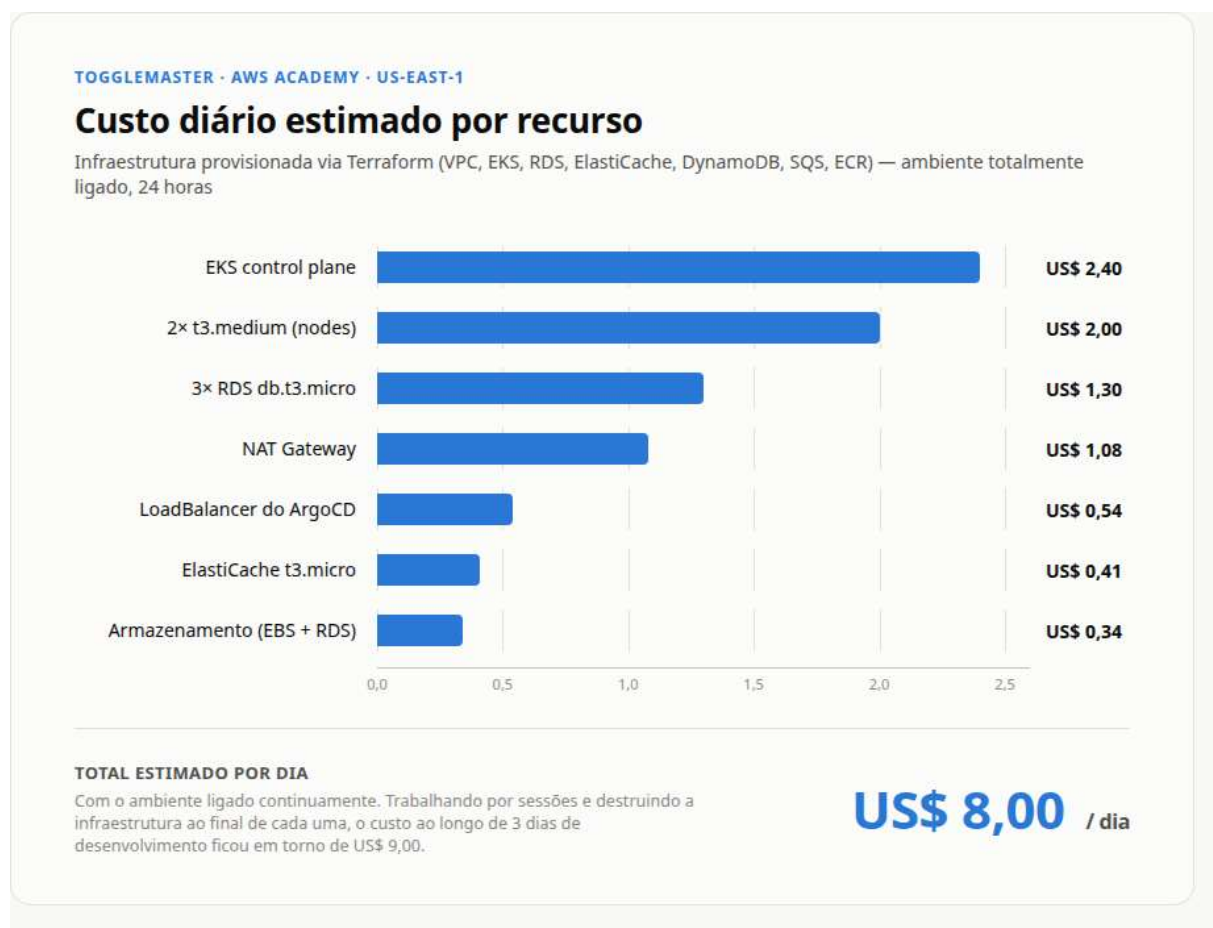
6.14. Race condition: 5 pipelines empurrando para a main simultaneamente

Ao disparar todos os 5 microsserviços de uma vez (um único commit alterando todas as pastas), cada pipeline tenta fazer push do commit de atualização de tag na main ao mesmo tempo. Apenas o primeiro sucede; os demais falham com non-fast-forward porque o HEAD já avançou. Não é um problema real do dia a dia — em produção

cada serviço deploya de forma independente, com diferentes commits e horários. Para fins de demonstração, disparar um serviço de cada vez (com ≥ 1 min de diferença entre commits) elimina a condição de corrida.

7. Estimativa de Custos da AWS

A figura abaixo reproduz a estimativa de custos publicada na documentação do projeto para a região us-east-1, com o ambiente totalmente provisionado. Rodando o ambiente ininterruptamente, o custo gira em torno de US\$ 8,00 por dia; trabalhando por sessões e destruindo a infraestrutura ao final de cada uma — rotina adotada pela equipe para conter o orçamento limitado do AWS Academy — o custo total ao longo dos três dias de desenvolvimento ficou em torno de US\$ 9,00.



Nota: esta tabela reproduz os valores documentados no repositório; não foi possível anexar aqui uma captura de tela ao vivo da AWS Pricing Calculator (calculator.aws) nesta rodada de geração do relatório.

8. Conclusão

O projeto cumpriu integralmente os requisitos técnicos da Fase 3: toda a infraestrutura do ToggleMaster — rede, cluster EKS, bancos de dados relacionais e não relacionais, cache, mensageria e repositórios de imagem — é declarada em Terraform componentizado, com estado remoto e lock, respeitando as restrições de IAM do AWS Academy. Os cinco microsserviços possuem pipelines de CI independentes, porém reutilizáveis, com um gate de segurança que efetivamente bloqueia o avanço da esteira diante de vulnerabilidades críticas, endereçando o problema real que motivou a Fase 3 — uma falha de segurança que chegou a produção sem ser detectada.

A adoção de GitOps com ArgoCD eliminou o kubectl apply manual e credenciais de cluster no pipeline de CI: o deploy passou a ser uma consequência automática de um commit no repositório de manifestos, com histórico completo, rollback trivial via git revert e sincronização contínua verificável na interface do ArgoCD. Combinados, IaC, DevSecOps e GitOps resolvem, de forma auditável e reproduzível, os quatro problemas operacionais que a DevOps Solutions Inc. havia identificado ao final da Fase 2.

Por fim, a restrição de privilégios do ambiente de treinamento AWS Academy — sem permissão para criar recursos de IAM e com orçamento limitado — foi o fator que mais influenciou decisões de arquitetura e a rotina operacional da equipe, exigindo o uso de data sources para a LabRole existente e uma disciplina de destruição do ambiente ao final de cada sessão de estudo para manter o custo sob controle.

9. Links de Entrega

- Repositório : <https://github.com/emilymarc/togglmaster-fase3/tree/main>
- Vídeo de demonstração: <https://youtu.be/TFgtYMbo-XI>